

SQL Databases

1. Overview

SQL databases are relational database systems that store data in structured **tables** consisting of rows and columns.

They are designed around:

- Tables
- Relationships between tables
- Primary keys
- Foreign keys
- Structured Query Language (SQL)
- ACID transactions

Examples:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database

2. Core Characteristics

A. Tables

Data stored in structured form.

Example:

Users Table

user_id	name	email
101	Samarth	sam@email.com

Orders Table

order_id	user_id	amount
5001	101	1200

B. Relationships

Tables can be linked logically.

None

`Users.user_id → Orders.user_id`

One user can have many orders.

3. Primary Key

Uniquely identifies a row.

Example:

None

`Users.user_id`

`Orders.order_id`

Rules:

- Unique
- Not null
- Indexed usually

4. Foreign Key

Used to reference another table's primary key.

Example:

None

`Orders.user_id` references `Users.user_id`

Ensures referential integrity.

Cannot create order for non-existing user.

5. ACID Properties

SQL systems are known for transactions with ACID guarantees.

Property	Meaning
Atomicity	All succeed or none
Consistency	Valid data rules maintained
Isolation	Concurrent transactions safe
Durability	Committed data survives crash

6. Example Transaction

Money transfer:

None

Deduct ₹100 from A

Add ₹100 to B

Commit

If failure mid-way → rollback.

Very important for banking/payments.

7. Why SQL Is Popular

Strong consistency

Reliable transactions

Rich query language

Joins across tables

Mature ecosystem

Great for structured data

8. SQL Query Examples

SQL

```
SELECT * FROM users;
```

SQL

```
SELECT * FROM orders WHERE user_id = 101;
```

SQL

```
SELECT u.name, o.amount
```

```
FROM users u
```

```
JOIN orders o ON u.user_id = o.user_id;
```

9. Best Use Cases

Use SQL when you need:

- Payments
- Orders
- Inventory
- User accounts
- ERP systems
- Banking systems
- Strong consistency
- Multi-row transactions

10. SQL vs NoSQL

Feature	SQL	NoSQL
---------	-----	-------

Schema	Fixed/structured	Flexible
Joins	Strong support	Limited/varies
Transactions	Strong	Limited/varies
Scaling	Harder historically	Easier horizontal
Consistency	Strong	Often tunable

11. Scaling SQL Systems

Vertical Scaling

Bigger server:

None

More CPU

More RAM

Faster SSD

Horizontal Techniques

- Read replicas
- Partitioning
- Sharding
- Caching
- Connection pooling

12. Read Replicas

Primary handles writes.

Replicas handle reads.

Useful for:

- Reporting
- Search pages
- High read traffic

13. Common Constraints

- UNIQUE(email)
- NOT NULL
- CHECK(amount > 0)
- FOREIGN KEY references

These help maintain data correctness.

14. Indexing

Indexes speed queries.

Example:

None

INDEX on email

INDEX on user_id

INDEX on created_at

Trade-off:

- Faster reads
- Slower writes
- More storage

15. When SQL Can Struggle

- Massive write scale without sharding
- Unstructured changing schema
- Global low-latency writes across regions
- Huge event streams

Then hybrid designs may help.

16. Interview Answer Example

Q: Why choose SQL here?

I'd use SQL because the system needs transactions, relationships between entities like users/orders/payments, referential integrity, and strong consistency.

17. Short Revision Notes

None

SQL DB:

- Tables
- Rows/columns
- PK/FK
- Joins
- ACID transactions

Best for:

payments, orders, structured business data

18. One-Line Summary

SQL databases are relational systems that use tables, keys, and ACID transactions to manage structured data with strong consistency and reliable relationships.